CSCI 4061 - Lab5

10/12/2026

beaver



University of Minnesota
Driven to Discover®

Shared Memory

- Another method for interprocess communication, other than pipe.
- Processes communicate by read/write a shared (physical) memory region.
- A “low-level” way of IPC, no built-in locks, no synchronization.
- No need to be in the same process tree. (via using shm_open and identical names)

Shared Memory Example

```
int* shm_ptr = mmap(NULL, shm_size, prot, flags, -1, 0);
pid_t child_pid = fork(); // memory region now shared across processes
```

create & share memory

```
if (child_pid == 0) {
    shm_ptr[0]=10;
    munmap(shm_ptr, shm_size);
    return 1;
} else {
    shm_ptr[1]=20;
    wait(...);
}
```

Operate on shared memory

```
printf("0: %d, 1: %d\n", shm_ptr[0], shm_ptr[1]); // 0: 10, 1: 20
```

access shared memory

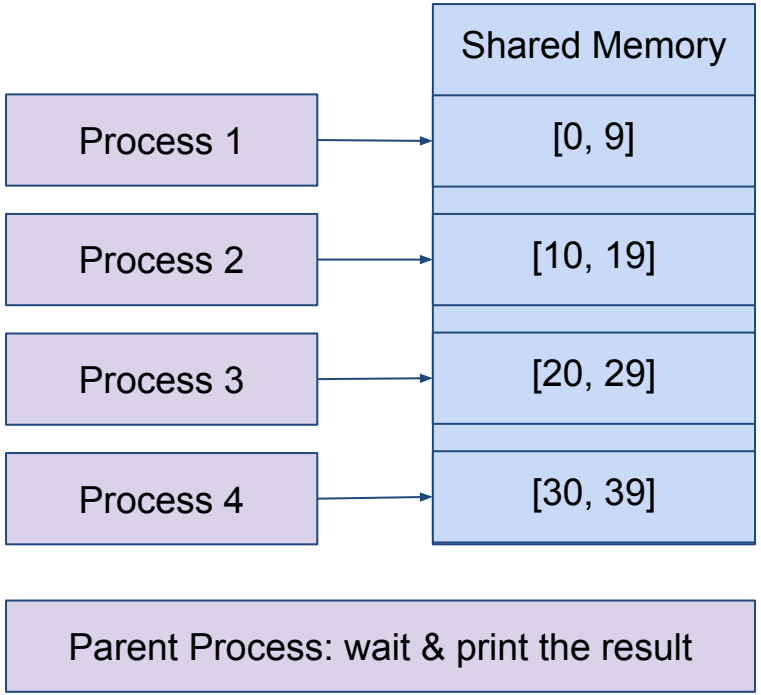
```
if (munmap(shm_ptr, shm_size) != 0) {
    perror("munmap");
    return 0;
}
```

Unmap the shared region (in both processes)

Today's Code

```
int numbers[40] = {
   75, 3,   99, 36, 47, 88,
93, 93, 0,  96, 63, 27, 12,
67, 75, 22, 34, 41, 61, 48,
   13, 100, 86, 15, 87, 35,
19, 92, 59, 70, 73, 65, 35,
35, 37, 44, 3,  20, 37, 27,
};
```

See QUESTIONS.txt for details.



Function needed for today's code

```
// map a memory region to the current process
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);

// addr: preferred starting address, set to NULL to let the kernel choose
// length: length of the memory region
// port(protection): PROT_READ | PROT_WRITE for a RW memory region.
// flags: property of the memory region, MAP_SHARED | MAP_ANONYMOUS for a
anonymous shared memory.
// fd and offset: set to -1 and 0.
// Return: start address of the memory region.


// remove memory mappings
int munmap(void *addr, size_t len);

// addr: start address of a memory region
// len: length of the removing memory region
```